

FROM MODEL INTELLIGENCE TO GOVERNED SYSTEM CONTROL

FROM AIRSPACE TO MARKETS: A GOVERNED AUTONOMOUS CONTROL ARCHITECTURE

A Synthetic Multi-Airport Simulator and Its Transfer to Finance

A pedagogical and academic study of how world modeling, perception, prediction, system analysis, bounded language-model judgment, deterministic validation, execution, physical dynamics, contingencies, observability and feedback combine into a governed autonomous control system—and how the same architecture can extend an already autonomous multi-market, multi-asset trading platform.

CENTRAL PROPOSITION

Autonomous intelligence is not an LLM. It is a closed-loop architecture of complementary forms of intelligence. A language model can add supervisory judgment, but authority remains with explicit rules, validators, execution constraints and feedback. The air-traffic sandbox makes this distinction visible; the financial transfer preserves it by adding judgment to an already autonomous trading stack rather than replacing deterministic market control.

Alejandro Reynoso

October 2026

AI for Financial Practitioners / Autonomous Systems / Control Architecture

Pedagogical Research Monograph

Educational simulator. The airspace, priorities, conflict thresholds, weather, wind, performance values and handoff rules are synthetic teaching constructs and are not operational Mexican air-traffic-control procedures or standards. The notebook is designed to explain autonomous-system architecture and its transfer to finance.

Abstract

This monograph studies a synthetic autonomous approach-control simulator built around sixty IFR aircraft interacting across MEX, NLU, and TOL. The simulator is not an aviation tool; it is a systems laboratory. Its object of study is the architecture of autonomy when the controlled object is no longer a single machine but an interacting population operating under shared constraints, uncertain forecasts, environmental shocks, and bounded authority.

The notebook develops fourteen architectural layers. A computational substrate establishes reproducibility and a single controlled interface to Claude Haiku 4.5. A world model creates airports, heterogeneous aircraft, weather, wind, and infrastructure state. Perception converts the evolving world into structured observations. Prediction projects short-horizon trajectories and identifies synthetic conflict risk. System analysis converts individual trajectories into congestion and sequencing information. A harness shapes a finite action space and independently validates every proposal. Context construction produces a compact situation packet. Claude contributes bounded supervisory judgment but has no direct execution authority. Validated interventions are translated into targets, physical dynamics determine the actual response, exogenous shocks modify the environment, and a closed feedback loop reconstructs the state. Telemetry, audit records, and radar-style visualization make the system observable.

The central claim is that **autonomy is an architecture, not a model call**. Reasoning and authority are distinct design problems. The LLM proposes; deterministic governance authorizes; execution acts; the world responds; the system observes again. This separation allows graceful degradation, auditability, and independent control of model capability and operational authority.

The final section transfers the architecture to finance. A multi-market, multi-asset trading system has the same structural problem: many interacting positions, venues, signals, liquidity states, constraints, and shocks evolve simultaneously. The relevant extension is not to replace an existing autonomous trading engine with an LLM, but to add a bounded judgment layer to an already automated control stack. The resulting design combines fast deterministic market control with slower supervisory judgment, while preserving risk limits, compliance rules, execution constraints, observability, and human authority.

COMPANION NOTEBOOK

The executable source of truth for this monograph is the Jupyter/Google Colab notebook referenced in the project as @TOPIC_784 SELF DRIVING ATC MMMX.ipynb. The monograph follows the notebook's 14-layer architecture and uses the supplied infographics in their corresponding sections.

Introduction: autonomy becomes system governance

The progression from a self-driving vehicle to an autonomous airspace simulator changes more than scale. It changes the nature of the control problem. A single vehicle has one principal body, one trajectory, and a comparatively compact state. A multi-aircraft environment contains simultaneous trajectories, shared infrastructure, heterogeneous performance, congestion, sequencing pressure, weather, wind, and interactions in which an intervention on one object changes the risk faced by another. The controlled object is therefore no longer the individual machine. It is the **system**: a changing configuration of agents, resources, constraints, forecasts, decisions, and consequences.

This change in the controlled object requires a broader definition of intelligence. Intelligence in an operational autonomous system is not a scalar property that can be assigned to a model after a benchmark. Nor is it exhausted by linguistic fluency, predictive accuracy, or the apparent sophistication of a single recommendation. It is a **multidimensional systems capability**. The relevant dimensions include the ability to perceive the environment, construct a coherent state, anticipate future conditions, identify interactions, exercise contextual judgment, select from a bounded action space, validate proposed actions, execute them reliably, observe their effects, preserve traceability, reproduce prior conditions, and remain governable when components fail or the environment departs from expectation. No one dimension can safely substitute for the others. Excellent prediction with poor execution does not create an intelligent controller; persuasive judgment without validation does not create an authorized action; and fast execution without reliable perception merely accelerates error.

The dimensions are also interdependent. Perception determines what evidence is available to judgment. Prediction determines whether the system reacts only to present conditions or anticipates developing ones. The harness determines which actions can be represented and considered. Governance determines which of those actions are admissible. Execution determines whether an authorized decision is translated into a well-formed intervention. Physical dynamics determine whether the intervention produces the intended state. Traceability determines whether operators can reconstruct why the system acted, and reproducibility determines whether researchers can distinguish a stable architectural result from an accidental run. Intelligence emerges from the disciplined composition of these functions rather than from the isolated strength of any one component.

The source notebook makes this transition explicit. Its governing sequence is:

World → Perception → Prediction → System Analysis
 → Context → Judgment → Validation
 → Execution → Dynamics → Feedback

Each stage solves a different problem. The world model determines what exists. Perception extracts a usable current state. Prediction introduces time and uncertainty. System analysis identifies interactions that cannot be understood by inspecting aircraft independently. Context engineering compresses the state into a decision packet. Claude Haiku 4.5 is used as a bounded supervisory reasoner. A deterministic harness then decides whether any proposal is admissible. Execution changes targets, not reality itself; the dynamics layer determines how the aircraft actually responds. Environmental shocks can change the world independently of the controller. Feedback forces the system to observe those consequences and decide again. The sequence is therefore not a linear software pipeline that ends after a model response. It is a closed loop in which every intervention becomes new evidence and every cycle can reveal a mismatch among intended action, authorized action, executed action, and realized outcome.

For analytical purposes, the architecture can be grouped into three broad domains: *perception*, *judgment*, and *execution*. Perception includes the sensors, data transformations, state estimates, forecasts, and aggregate measures that make the system legible. Judgment interprets that evidence, weighs competing objectives, recognizes unusual combinations of conditions, and proposes a limited response. Execution translates an accepted proposal into target changes and ultimately into movement in the world. Yet these three domains are insufficient unless they are enclosed by a fourth domain: *governance*. Governance includes the harness that constructs candidate actions, the schema that restricts model output, the validator that applies deterministic rules, the authority boundaries that prevent direct model control, and the fallback behavior that preserves operation when the judgment component is unavailable. Harnessing is thus not merely a prompt-engineering technique. It is the executable institutional design through which intelligence is given a defined role, a limited vocabulary, and a controlled path to action.

This distinction separates reasoning from authority. A model may be capable of explaining a complex situation while remaining unauthorized to change it. Conversely, a deterministic component may possess narrow authority even though it has little capacity for contextual interpretation. The architecture deliberately combines these asymmetries. The language model contributes flexible supervisory judgment where ambiguity and interacting considerations make fixed rules brittle. The harness and validator retain authority where safety, compliance, consistency, and auditability require explicit behavior. This division permits capability to improve without allowing permissions to expand automatically. A newer or more capable model can be evaluated inside the same action space and under the same

constraints before any governance boundary is changed.

This architecture is consistent with a long tradition of control, human-machine systems, and situation-awareness research in which perception, decision, automation, feedback, and authority must be treated as separable design objects [2; 3; 4; 5]. The distinctive pedagogical contribution of the notebook is to insert a modern generative model into this architecture without confusing generative reasoning with control authority.

CENTRAL THESIS

A large language model can improve the *judgment* of an autonomous system without becoming the system's controller. Capability can be expanded while authority remains narrow, explicit, deterministic, and auditable.

Traceability and reproducibility are essential dimensions of this account of intelligence, not administrative features added after the controller is built. Traceability connects an outcome to the state that was perceived, the forecast that was available, the candidate actions that were constructed, the judgment that was proposed, the rules that accepted or rejected it, and the intervention that was executed. An audit record must therefore preserve more than a model's final sentence. It should preserve inputs, timestamps, model and rule versions, candidate sets, validation results, fallback states, target changes, and observed consequences. This evidence makes it possible to distinguish a poor forecast from a poor recommendation, a governance failure from an execution failure, and an adverse but admissible outcome from unauthorized behavior.

Reproducibility serves a related but distinct purpose. A reproducible system can reconstruct the relevant initial conditions, configurations, random seeds, model settings, and decision path well enough to repeat an experiment or explain why exact repetition is impossible. In a stochastic environment, reproducibility does not mean that every future trajectory must be identical. It means that the system's designed behavior can be tested under controlled conditions and that differences between runs can be attributed to identified sources. This is indispensable for scientific evaluation, incident analysis, model comparison, and governance review. Without it, claims of intelligence may reflect hidden state, favorable randomness, or undocumented configuration changes.

These principles are especially important for finance. Modern trading and investment systems are already highly automated. Market-data collection, signal computation, risk checks, order slicing, venue routing, position accounting, and kill switches do not need to be reinvented as language-model behavior. The more interesting question is how to add judgment to these systems without dismantling the mechanisms that make them fast, reliable, and governable. The airspace sandbox provides a concrete way to reason about that transfer because it exposes the same abstract problem in an intuitively physical setting: many heterogeneous entities compete for constrained resources; local actions alter global conditions; forecasts are uncertain; external shocks change the feasible set; and errors propagate through interaction.

The proposed transfer is therefore **systems-level transfer**, not the transfer of aviation procedures into markets. Aircraft are not assets, runways are not exchanges, and synthetic separation thresholds are not financial risk limits. The transferable object is the architecture by which a complex operational system converts observations into governed interventions. An aircraft trajectory can correspond abstractly to the evolving exposure of a position; shared airspace to common liquidity and risk capacity; congestion to crowded positioning or limited market depth; weather to an external volatility or liquidity shock; sequencing to capital allocation across competing opportunities; handoff to a change of venue, strategy, mandate, or risk owner. These are functional correspondences, not claims of domain equivalence.

Learning at the systems level asks which structural relationships remain useful when the physical meaning of the variables changes. Both domains require a world model that identifies entities and constraints, a perception layer that creates trusted state, prediction that estimates near-term evolution, system analysis that detects interactions, bounded judgment that interprets competing objectives, deterministic governance that enforces hard limits, execution that respects operational constraints, and feedback that measures realized effects. Both also require escalation paths, graceful degradation, versioned policies, and records that allow decisions to be reconstructed. This common structure permits architectural knowledge gained in a physical operational system to inform the control of investment flows and decisions without pretending that the empirical laws, timescales, or consequences are identical.

The transfer also clarifies the role of an investment decision. A decision is not complete when a model recommends buying, selling, hedging, reallocating, or waiting. It becomes operational only after the recommendation has been mapped to eligible instruments and accounts, checked against exposure and liquidity limits, reconciled with existing orders and positions, approved under the relevant mandate, translated into an execution schedule, and monitored for realized market impact. The realized portfolio then changes the next perception and judgment problem. Finance, like airspace control, is consequently a feedback system in which the act of control modifies the state being controlled.

This monograph uses the synthetic simulator to make that architecture visible layer by layer. Its objective is not to argue that a language model should direct aircraft or capital. It is to show how heterogeneous forms of intelligence can be assembled into a controlled whole: numerical perception, statistical anticipation, relational system analysis,

language-enabled judgment, rule-based authorization, constrained execution, dynamic response, and auditable feedback. The resulting framework treats intelligence as a property of the governed loop. It is this loop—rather than any isolated model, prompt, or forecast—that can be evaluated, improved, and responsibly transferred from one complex domain to another.

The experiment: a synthetic multi-airport control laboratory

The simulator contains sixty IFR aircraft distributed across three synthetic airport objects representing MEX, NLU, and TOL. Airport weights are stylized at 28:7:4. Aircraft are heterogeneous: regional, narrow-body, wide-body, cargo, and military performance profiles have different synthetic speeds, climb rates, and descent rates. Passenger, military, and cargo classes receive scenario-specific priority values used only to make sequencing visible. Weather cells can activate, move, intensify, and dissipate. Wind evolves continuously and can change simplified runway modes. Arrivals and departures coexist, and aircraft eventually leave the terminal system through synthetic tower or en-route handoffs.

Reproducibility is built into the experiment. A fixed seed creates the same starting fleet. Airport and weather reset functions return the world to a known initial state before the integrated run. The notebook therefore distinguishes between a system that can be rerun and one whose results depend on hidden state left behind by a previous execution.

The external model connection is also deliberately isolated. All Claude calls pass through one gateway using the model `claude-haiku-4-5-20251001`. The notebook sets a strictly positive temperature value of 0.2 and guards it with an assertion. This is an implementation robustness choice rather than a conceptual property of autonomous control. More important is the architectural rule that the model interface is centralized: later cells cannot silently introduce a second configuration or a different authorization path.

Layer	Cell(s)	Function	Control question
1	2–5	Setup and architecture	What computational substrate and authority model exist?
2	6–10	World/environment	What objects, capabilities, weather, wind, and infrastructure define the state space?
3	11	Perception	What is happening now?
4	12	Prediction	What may happen next?
5	13	System analysis	Where is congestion or sequencing pressure emerging?
6	14–16	Harness and rules	Which actions may be considered, and which proposals are admissible?
7	17	Context construction	What compact evidence packet should supervisory judgment receive?
8	18	Judgment	Which bounded interventions should be proposed?
9	19	Execution	Which accepted proposals alter target state?
10	20	Dynamics/handoff	How does the world physically respond?
11	21	Environmental shocks	What changes independently of the controller?
12	22	Feedback	How is the next control cycle created?
13	23	Observability	What can a human inspect?
14	24	Integrated run	Does the architecture operate coherently end to end?

The experiment is intentionally synthetic. The thresholds, distances, priorities, rates, and runway abstractions are chosen to reveal system behavior in a compact notebook. They are not operational aviation standards. This matters because the monograph’s objective is not to infer policy from a toy simulation. It is to extract a transferable control architecture.

Layer 1: computational substrate, reproducibility, and the authority model

Cells 2–4 establish the laboratory. Dependencies are intentionally small: NumPy, pandas, matplotlib, and the Anthropic client. Global parameters define the seed, fleet size, number of ticks, handoff radius, supervisory interval, and distance scale. This is engineering rather than intelligence, but autonomous intelligence cannot be audited if its environment is not stable enough to reproduce.

The Claude interface is a particularly important boundary. Authentication is separated from reasoning logic, and `claude_text()` becomes the single gateway for model calls. The architecture therefore has one place where model identity, token limits, timeout behavior, retries, and temperature are controlled. This prevents configuration drift and gives the larger system a clear dependency boundary: if the external model is unavailable, the simulator can detect that condition and continue with deterministic fallback.

Layer 1: system setup and initialization, Cells 2–4.

Cell 5 then states the architecture in institutional terms. The LLM is not a free-form controller. It occupies one role inside a governed loop. The decisive sentence is: **reasoning proposes; deterministic governance authorizes; execution acts**. That sentence converts the simulator from an “agent demo” into a control architecture.

The system also contains two time scales. The fast loop advances weather, wind, perception, numerical prediction, validation, execution, and dynamics at every tick. The slower supervisory loop calls the generative reasoner periodically or when the state justifies reconsideration. This is a powerful general design principle. Expensive or semantically rich reasoning should not necessarily run at the frequency of the physical or market process being controlled.

Layer 1 also establishes the provenance needed for meaningful experimentation. A fixed random seed, centralized configuration, explicit model identifier, and controlled initialization make the resulting behavior attributable to a known setup. Without those provisions, two runs that appear to test the same architecture may begin from different fleets, use different inference settings, or inherit state from earlier notebook execution. Reproducibility is therefore part of the control design: it creates a stable reference against which changes in policies, predictors, prompts, or validators can be evaluated.

The authority model defined here is equally foundational. Components receive permissions according to function rather than apparent intelligence. Numerical routines may update forecasts; the supervisory model may propose from a supplied menu; the validator may authorize or reject; and the execution layer may alter only specified target variables. These interfaces create a chain of custody for decisions. They also support replacement: a new forecasting method or language model can be inserted behind the same interface without silently acquiring additional authority. Layer 1 thus supplies both the computational substrate and the constitutional structure within which every later layer operates.

Cell 5: the architecture of the governed autonomous approach-control system.

TRANSFER PRINCIPLE 1 – KEEP JUDGMENT AND AUTHORITY SEPARATE

In finance, a supervisory model may interpret regime changes, cross-asset tensions, or competing objectives. It should not therefore acquire the ability to send arbitrary orders. Order authority should remain in deterministic execution and risk-control layers whose permissions can be tested independently of model quality.

Layer 2: building a world before trying to control it

Cells 6–10 create the environment on which later cognition depends. The notebook begins with three airport objects. Each airport has a code, synthetic position, traffic weight, wind state, and simplified runway mode. The point is not geographical fidelity. It is to create shared infrastructure whose state matters to multiple agents simultaneously.

Heterogeneous performance is introduced next. Regional, narrow-body, wide-body, cargo, and military categories receive different synthetic motion parameters. This prevents the controller from treating all aircraft as interchangeable points. Heterogeneity is essential in systems problems because a common instruction can have different consequences depending on the controlled object's capabilities.

The fleet generator then creates sixty IFR aircraft. Arrivals are initialized farther from their airport and point toward it; departures begin close to the airport and target higher altitudes. NLU is given a mixed passenger, cargo, and military traffic context, while the other airports use passenger and cargo traffic. The priority mapping is explicitly scenario-specific and pedagogical.

Weather becomes the first important exogenous variable. Weather cells have position, radius, velocity, severity, and active state. They can move and change size. Wind introduces a second environmental variable and can change runway mode. The lesson is fundamental: an autonomous system does not control all variables that determine the quality of its decisions.

The value of the world model lies in its explicit ontology. Aircraft, airports, weather cells, infrastructure modes, and handoff status are represented as different kinds of objects with different state variables and transition rules. That distinction prevents the controller from reducing the environment to an undifferentiated table of numbers. It can ask which attributes belong to an aircraft, which constraints are shared through an airport, and which disturbances originate outside the controlled population. Such structure is necessary for causal interpretation: an aircraft may slow because of an authorized target change, while a queue may grow because weather reduced effective capacity.

The model must also represent what the controller does not know perfectly. Although the notebook uses a fully observable synthetic environment for pedagogical clarity, a production system would attach timestamps, quality indicators, uncertainty ranges, and missingness states to important variables. The conceptual lesson remains the same: the world model should identify the entities, relationships, capabilities, and disturbances whose evolution matters to control. If a relevant constraint does not exist in the state representation, no later reasoning layer can reliably govern it.

Layer 2: world and environment model, Cells 6–10.

Formally, let the complete environment state be x_t . It contains aircraft states, airport states, weather, and other shared variables. External disturbances enter through ε_{t+1} , so the world evolves as

$$x_{t+1} = F(x_t, a_t, \varepsilon_{t+1}),$$

where a_t represents executed interventions. This equation already tells us why a static recommendation is insufficient: even with $a_t = 0$, the state can change because $\varepsilon_{t+1} \neq 0$.

TRANSFER PRINCIPLE 2 – MODEL THE WORLD AS A STATE, NOT AS A PROMPT

A multi-market trading system should maintain explicit state for venues, order books, positions, cash, collateral, borrow availability, volatility, correlations, latency, market status, and connectivity. The LLM should consume a governed representation of that state; it should not reconstruct the trading world from prose each time it is called.

Layer 3: perception as structured state construction

Cell 11 turns simulation objects into an operational perception. For each active aircraft, the perception layer records callsign, airport, phase, traffic class, altitude, speed, distance to the assigned airport, weather exposure, hold state, and priority. The result is a pandas DataFrame: a standardized current-state table.

This step is easy to underestimate. In many AI demonstrations, raw environment and model context are collapsed into one object. Here they are separated. The world may contain more information than the reasoner needs, and the reasoner should not be responsible for deciding how basic measurements are calculated. Distance to airport, for example, is computed deterministically. Weather exposure is derived from geometry. Inactive or handed-off aircraft are filtered out before higher reasoning begins.

Perception can be written as

$$o_t = H(x_t),$$

where H is a deterministic observation function and o_t is the structured state visible to downstream layers. Good perception is therefore partly a schema-design problem: choose fields that make later prediction, risk analysis, and judgment explicit and testable.

Layer 3: perception, Cell 11.

The relationship to situation awareness is direct. Endsley distinguishes perception of elements, comprehension of their meaning, and projection of future status [3]. The notebook deliberately distributes those functions across separate computational layers. Cell 11 performs the first transformation; Cells 12 and 13 add projection and system-level meaning.

Structured perception also establishes semantic consistency. A field such as `distance_to_airport` must have one definition, one unit convention, and one update rule across all aircraft and ticks. The same requirement applies to phase, active status, weather exposure, and priority. If downstream components interpret these fields differently, the system may appear computationally functional while making incoherent decisions. Schema definitions, data types, permissible values, and freshness expectations are therefore part of perception rather than mere implementation detail.

Data quality should be treated as perceived state in its own right. A production controller must know whether an observation is current, delayed, estimated, conflicting, or unavailable. It may then reduce authority, request confirmation, use a conservative fallback, or decline to intervene. This principle is transferable to finance, where a price can be numerically valid but stale, a position can be correctly formatted but not reconciled, and a volatility estimate can be current for one venue but obsolete for another. A robust perception layer represents both the measurement and confidence in the measurement.

The separation between world and observation also improves testing. The simulator can vary the observation function while holding the underlying world constant, making it possible to study whether errors originate in the environment, in measurement, or in state construction. Unit tests can verify deterministic calculations, filtering, and schema invariants before any generative judgment is invoked. This prevents a common analytical mistake in which an incorrect recommendation is attributed to the reasoner even though the reasoner received an incomplete or inconsistent state.

Perception is consequently an active design choice about operational relevance. Including every available variable can obscure critical evidence and increase processing cost, while omitting a constraint can make safe action impossible. Layer 3 creates the canonical present from which the rest of the system reasons. Its quality is measured not by the volume of data collected but by whether it provides a timely, coherent, traceable, and decision-relevant account of the controlled environment.

Layer 4: prediction turns state into anticipatory control

Cell 12 introduces the future. The function `predict()` projects each active aircraft over a short horizon using its present speed, heading, and movement toward target altitude. The function `conflicts()` compares projected aircraft pairs and identifies synthetic conflict conditions when projected lateral and vertical distances cross pedagogical thresholds.

Prediction changes the nature of control. A reactive controller waits until the present state is already problematic. An anticipatory controller asks what the state is likely to become before choosing an intervention. If P_h is a horizon- h prediction operator,

$$\hat{x}_{t+h} = P_h(o_t).$$

The forecast need not be perfect. It needs to be sufficiently informative to improve current action relative to a purely reactive alternative.

Layer 4: prediction and synthetic conflict detection, Cell 12.

The notebook uses simple geometry rather than an LLM because that is the appropriate tool for the job. This is a broader architectural principle: generative intelligence should not replace numerical methods that are cheaper, more transparent, and better matched to the computation. A mature autonomous system combines forms of intelligence rather than forcing every task through one model.

Prediction introduces several design choices that must remain explicit. The forecast horizon determines which developing interactions are visible; assumptions about constant heading or speed determine how rapidly uncertainty grows; and thresholds determine which projected proximities are elevated for attention. A short horizon may miss slowly developing congestion, while a long horizon may generate excessive warnings from trajectories that will change before the forecast is realized. The appropriate horizon is therefore linked to the response time of the controlled system and the interventions available to it.

Forecasts should also retain their status as estimates rather than facts. The predicted state is conditional on current observations, simplified dynamics, and assumptions about other agents. A governed controller can respond by attaching confidence, testing multiple scenarios, or requiring stronger evidence for more consequential interventions. Even in this simplified notebook, the separation between x_t and $\hat{x}_{t+h}$ prevents future projections from being confused with observed state. That semantic boundary becomes critical when predictions influence capital allocation or safety-relevant action.

Evaluation of Layer 4 should focus on decision usefulness as well as statistical accuracy. A forecast can have low average error yet fail to identify the rare interactions that matter most. Conversely, an imperfect forecast may still be valuable if it consistently provides enough lead time for a modest, safe intervention. Useful metrics therefore include warning lead time, false and missed conflict indications, stability across adjacent ticks, calibration of uncertainty, and the downstream effect on authorized actions.

In financial transfer, the same logic discourages a single undifferentiated forecast of “the market.” Separate models may estimate volatility, liquidity, correlation, fill probability, or market impact over horizons matched to their control decisions. The judgment layer then receives these forecasts as evidence, while governance decides how much authority each estimate may support. Layer 4 turns the autonomous architecture from a description of current state into a mechanism for controlled anticipation.

TRANSFER PRINCIPLE 3 – FORECAST WHAT CONTROL NEEDS, NOT EVERYTHING THAT CAN BE FORECAST

In trading, the equivalent forecast layer may estimate short-horizon volatility, liquidity, market impact, spread, correlation breaks, fill probabilities, or inventory risk. These models can remain statistical or mechanistic. Generative judgment is most useful after such forecasts have been converted into a structured decision problem.

Layer 5: system analysis, congestion, and sequencing

Cell 13 converts a population of individual aircraft into system-level information. It counts arrivals within a synthetic 45-unit radius and departures within 35 units for each airport. Arrival queues are sorted by scenario priority and distance, and the first ten callsigns form a visible sequence.

This layer illustrates a key difference between object intelligence and system intelligence. Perception and prediction may be accurate for every individual aircraft while still failing to answer a higher-level question: where is the system becoming congested? Aggregation, queue construction, resource pressure, and interaction density are distinct analytical tasks.

Let

$$s_t = A(o_t, \hat{x}_{t+h}),$$

where A is a system-analysis operator. Its output is not another trajectory. It is a statement about the structure of interactions: congestion, queue pressure, concentration, and sequencing needs.

Layer 5: system analysis, congestion, and sequencing, Cell 13.

This distinction becomes even more important in finance. A trading engine can understand each position independently and still fail to understand portfolio crowding, factor concentration, cross-venue liquidity dependence, or a common funding constraint. System analysis must therefore operate above the instrument level.

Layer 5 creates derived variables that exist only at the level of the population. A queue is not an attribute of one aircraft; it is an ordered relationship among several aircraft competing for shared capacity. Congestion is not merely a large number of objects; it is a condition in which demand, geometry, timing, and infrastructure constraints interact. By constructing these variables explicitly, the notebook gives later judgment a system-level vocabulary rather than expecting the model to infer global structure from dozens of isolated records.

Sequencing also reveals that control requires an objective, even when the objective is simple and pedagogical. The notebook combines priority and distance to produce a visible order. A more realistic system would have to reconcile several objectives: safety margins, delay, fuel use, fairness, infrastructure utilization, downstream capacity, and exceptional status. Making the sequence explicit permits these tradeoffs to be inspected and challenged. It also prevents a hidden or accidental ordering from becoming de facto policy.

System analysis can identify externalities created by local decisions. Slowing one aircraft may resolve a projected pairwise conflict but increase queue pressure at another location; releasing a hold may improve one trajectory while consuming scarce arrival capacity. The analytical layer should therefore recompute aggregate conditions after interventions rather than treating pairwise resolution as sufficient. This is the point at which the controlled object becomes the network of interactions rather than the set of independent agents.

In finance, analogous derived states include net and gross exposure, factor loading, correlated drawdown risk, liquidity concentration, shared collateral consumption, crowded exits, and dependency on a common venue or data source. None of these can be understood by inspecting a proposed order alone. A set of individually compliant trades can collectively breach a risk appetite or create an unmanageable liquidation profile. System analysis must identify these joint conditions before judgment allocates capital or changes exposure.

The output of Layer 5 should therefore be interpretable, versioned, and linked to the observations from which it was computed. Operators need to know not only that an airport or portfolio is labeled congested, but which counts, thresholds, rankings, and constraints produced that label. This traceability makes aggregate intelligence auditable and allows the institution to distinguish a data problem from a policy problem. Layer 5 is the bridge between accurate knowledge of individual objects and intelligent control of the system as a whole.

Layer 6: the harness as executable governance

Cells 14–16 create the system’s control boundary. Cell 14 provides the conceptual rule: safety is not a prompt instruction. It is executable governance. Cell 15 generates a finite candidate action menu. Cell 16 validates returned proposals against aircraft identity, active status, action vocabulary, numeric envelopes, weather checks, phase restrictions, and exact membership in the current candidate set.

The allowed actions are intentionally narrow: MAINTAIN, SPEED, VECTOR, ALTITUDE, HOLD, and RELEASE. Candidate generation changes by phase. Arrivals may be slowed, vectored, moved in altitude, held, or released. Departures receive a different bounded menu. Duplicate choices are removed.

The key point is the difference between **action shaping** and **validation**. The model is not asked to invent an action and hope that a prompt keeps it reasonable. Deterministic code first shapes the action space. A later deterministic function checks whether the proposal still conforms.

$$\mathcal{C}_t = \mathcal{C}(x_t)$$

denotes the candidate set generated from the current state. If the supervisory system proposes p_t , then execution authority is

$$a_t = \begin{cases} p_t, & \text{if } V(p_t, x_t, \mathcal{C}_t) = 1, \\ \emptyset, & \text{otherwise.} \end{cases}$$

Layer 6: harness, bounded actions, and deterministic validation, Cells 14–16.

This is the most transferable governance idea in the notebook. Model quality and model authority should be independent. Upgrading the reasoner does not require widening the action envelope. Conversely, the institution can change permissions without retraining the model. This separation is consistent with safety-oriented systems engineering [5] and with contemporary AI risk-management frameworks that emphasize governance, mapping, measurement, and management across the system lifecycle [6].

The harness also makes policy testable. Each constraint can be exercised against boundary values, incompatible phases, inactive objects, stale candidates, and malformed outputs. Rejections become structured events rather than silent failures. This permits a governance team to measure which rules are binding, whether candidate generation is too narrow or too permissive, and whether a model repeatedly attempts unavailable actions. The resulting evidence can improve either the reasoner or the policy without confusing their responsibilities.

Because admissibility depends on current state, authorization must be time-sensitive. A proposal that was valid when context was constructed may become invalid after weather, traffic, or aircraft status changes. Exact membership in the current candidate set helps prevent an old proposal from being executed under new conditions. In higher-risk deployments, the same principle can be strengthened with state versions, expirations, idempotency keys, and approval tokens tied to a particular snapshot.

Governance should also include abstention and escalation. The absence of an admissible action is a legitimate outcome, not a software defect. When every candidate violates a hard constraint or available evidence is insufficient, the correct response may be to maintain state, invoke a deterministic contingency, or transfer authority to a human operator. Layer 6 therefore defines not only what the autonomous system may do, but the circumstances under which it must decline to act.

TRANSFER PRINCIPLE 4 – DEFINE THE FINANCIAL ACTION SPACE OUTSIDE THE LLM

For a trading system, deterministic candidate generation might encode maximum participation rate, order size, price bands, inventory limits, leverage, venue eligibility, locate availability, restricted lists, concentration caps, and kill-switch conditions. The LLM may choose among admissible actions; it should not define what “admissible” means.

Layer 7: context construction as an engineering discipline

Cell 17 builds the world-state packet for supervisory judgment. It starts from predicted conflicts, congestion, active weather, runway state, and a selected set of focus aircraft. It then attaches the bounded candidate actions available for those aircraft. The result is a compact JSON-compatible object rather than a dump of the entire simulator.

This layer deserves special emphasis because context engineering is often treated as prompt decoration. Here it is a state-compression problem. The packet must be sufficiently rich to support judgment, sufficiently small to be inspectable, and sufficiently structured to keep the reasoner inside the system's vocabulary.

The focus mechanism is itself instructive. Aircraft involved in predicted conflicts receive attention, as do aircraft near the top of arrival sequences. The packet therefore allocates scarce supervisory attention to objects that the deterministic layers have already identified as consequential.

$$c_t = C(o_t, \hat{x}_{t+h}, s_t, \mathcal{C}_t)$$

is the context constructor. The model receives c_t , not the raw world x_t .

Layer 7: context construction and the situation packet, Cell 17.

In institutional terms, context construction separates evidence preparation from decision judgment. A credit committee, investment committee, or trading supervisor should not need to reconstruct the firm's state from raw databases at the moment a decision is required. The system should produce a governed decision packet.

Context construction necessarily involves loss, so its selection policy must be visible and testable. Compressing sixty aircraft into a small focus set improves tractability, but it can omit an object whose relevance was not captured by the deterministic attention rules. The architecture should therefore record why each item was included, which summaries cover excluded objects, and whether any capacity or truncation limit was reached. This turns context selection into an auditable transformation rather than an invisible editorial step.

The packet should distinguish observations, predictions, system-level conclusions, constraints, and candidate actions. These categories have different epistemic status. An observed altitude is not the same kind of evidence as a projected conflict; a congestion count is not an instruction; and a candidate action is not yet authorized. Clear structure reduces the chance that supervisory judgment treats an estimate as a fact or a possible action as a command. It also simplifies validation because returned choices can be matched to exact identifiers and values.

Good context is temporally coherent. Fields should refer to a common state version or disclose their timestamps, particularly when the world changes quickly. A packet assembled from current positions, an older weather state, and stale candidate actions may be syntactically valid but operationally misleading. Versioning and freshness checks allow the execution boundary to determine whether the evidence on which a proposal relied is still applicable.

In finance, a governed decision packet might contain current exposures, recent changes, liquidity forecasts, binding limits, correlated risks, pending orders, scenario losses, and a finite set of permissible adjustments. It should omit secrets or irrelevant personal data and expose only the detail needed for the assigned role. Layer 7 therefore performs both cognitive compression and access control: it decides what supervisory intelligence can see, how that evidence is labeled, and which decisions the evidence is intended to support.

Layer 8: bounded supervisory judgment and graceful degradation

Cell 18 introduces Claude Haiku 4.5 as the judgment layer. The model is instructed to propose at most six interventions, use only candidate actions supplied for the relevant callsign, avoid inventing numerical values, prefer the smallest useful intervention, and return a strict JSON array containing callsign, action, value, and reason.

The model therefore operates as a chooser over a constrained set rather than as a free-form controller. In formal terms,

$$p_t = J(c_t),$$

but p_t still lacks execution authority. It is merely a proposal.

The deterministic fallback is equally important. If the Claude client is unavailable, the model times out, parsing fails, or another exception occurs, the simulator continues. The fallback examines predicted conflicts and selects modest speed adjustments for selected lower-priority aircraft. These fallback proposals still pass through the same validator.

Layer 8: bounded supervisory judgment with deterministic fallback, Cell 18.

This creates **graceful degradation**. The system can lose a sophisticated judgment component without losing the entire control loop. Such designs are preferable to architectures in which model connectivity is synonymous with system availability.

The use of a model to reason and then act through externally defined interfaces is related to the broader literature on reasoning-and-acting systems [9]. The notebook's contribution is to place that pattern inside a harder governance boundary: tools and actions are not merely available; they are shaped and authorized independently.

The judgment layer adds value where the state contains competing considerations that are difficult to reduce to a single fixed rule. It can compare several predicted conflicts, congestion conditions, priorities, and intervention costs, then recommend a small subset for attention. Its natural-language reason field can communicate the basis of a choice to operators, but that explanation is supporting evidence rather than proof that the action is correct. The operative decision remains the structured action and value submitted to validation.

Bounded judgment should be evaluated for consistency, selectivity, and restraint. A useful supervisor need not act on every call. It should recognize when maintaining the current plan is preferable, avoid oscillating among interventions across adjacent ticks, and reserve action for conditions where the expected benefit justifies disturbance to the system. The instruction to prefer the smallest useful intervention reflects a general control principle: limited changes are easier to reverse, attribute, and monitor than broad simultaneous changes.

The strict output schema supports traceability and fault containment. A proposal can be associated with the exact context packet, model version, prompt version, candidate menu, and generation time. Parsing failure then has a defined meaning and response rather than producing ambiguous partial execution. The validator does not infer intent from prose or repair an unauthorized value. This prevents linguistic plausibility from bypassing operational controls.

Graceful degradation must be tested deliberately. The fallback path should be exercised under missing credentials, timeouts, malformed JSON, empty responses, and model-service failures, not merely assumed to work. Its behavior should remain conservative and observable, with telemetry distinguishing model-generated proposals from fallback proposals. Layer 8 thereby demonstrates that generative judgment can enrich autonomous operation without becoming a single point of failure or an unbounded source of authority.

Layer 9: execution translates authority into target state

Cell 19 receives proposals and runs the deterministic validator again. Accepted actions update target speed, target heading, target altitude, or hold state. Rejected actions produce no state change. Every proposal, accepted or rejected, is written into an execution log with the validator reason.

The repeated validation is intentional. In a governed system, an earlier recommendation is not a permanent authorization. Execution is the final boundary at which the proposed action meets current state. This matters whenever state can change between reasoning and execution.

Layer 9: execution after validation, Cell 19.

The difference between a target and a realized outcome is also important. Cell 19 can change `target_speed`; it does not instantaneously change actual speed. That distinction belongs to the next layer and is essential for any real control architecture.

Execution is a transaction boundary. Before changing state, it should confirm that the proposal refers to the intended object, was authorized against an applicable state version, has not expired, and has not already been applied. Afterward, it should record the exact mutation and whether it succeeded. These properties prevent duplicate execution, stale action, partial updates, and ambiguity about whether a recommendation became operational. Even in a compact simulator, the accepted/rejected log establishes this chain of custody.

The layer also converts semantic decisions into machine state. A supervisory reason may discuss congestion or conflict, but the executor acts only on explicit fields such as target speed, heading, altitude, or hold status. This translation protects the controlled system from open-ended text and makes effects measurable. It also enables testing: each action type can be checked for its permitted variables, numerical range, and invariants independently of the model that proposed it.

Execution failures are different from validation failures and should be represented separately. A proposal may be admissible yet fail because an object disappeared, a dependency became unavailable, or a state update could not be committed. Robust systems report these conditions rather than treating authorization as evidence of completion. Where several updates must occur together, transactional or compensating behavior may be necessary to prevent an inconsistent intermediate state.

In finance, this boundary is where an approved allocation or risk response becomes orders, cancellations, participation schedules, or changes to execution parameters. Pre-trade controls should be rerun against current positions and market state because exposure, price, or liquidity may have changed since judgment. Order identifiers, parent-decision identifiers, timestamps, rejections, and acknowledgments create lineage from supervisory intent to venue interaction. Layer 9 therefore makes authority concrete while preserving the distinction among recommendation, authorization, submission, and eventual outcome.

TRANSFER PRINCIPLE 5 – EXECUTION SHOULD RECEIVE TARGETS, NOT STORIES

A financial judgment layer may propose “reduce technology beta” or “slow participation in venue X,” but the execution layer must convert the approved intention into explicit orders and constraints. Every order should be attributable to a validated decision, and every rejection should have a machine-readable reason.

Layer 10: dynamics, inertia, and handoff

Cell 20 converts accepted targets into physical response. Heading moves toward target heading with a bounded step. Speed moves toward target speed gradually. Altitude changes within aircraft-specific climb or descent limits. Arrivals not in hold reorient toward their airport; departures target higher altitude. Aircraft in hold rotate gradually. Position is then updated from speed and heading.

This layer makes an essential control point visible: decisions change intentions faster than the world can respond. The next state depends on dynamics, not on the instruction alone. This is why a controller must observe the consequences of its action rather than assume that the requested state was achieved.

Arrivals that satisfy synthetic distance and altitude criteria are handed to tower; departures that move sufficiently far and high are handed to en-route control. Handoff removes them from the active terminal population.

Layer 10: aircraft dynamics and handoff, Cell 20.

The same principle dominates trading. An order is not a fill. A desired portfolio is not an achieved portfolio. Market impact, queue position, spread, latency, partial fills, and other agents' behavior create a financial dynamics layer between decision and outcome [10; 11; 12]. Any autonomous trading architecture that collapses decision and realized state into one step will mismeasure both risk and control quality.

Dynamics introduce rate limits and path dependence. An aircraft cannot adopt an arbitrary heading, altitude, or speed instantaneously; its next state depends on its previous state and performance envelope. Consequently, two identical targets issued under different initial conditions can produce different trajectories and risks. The controller must reason not only about desirable terminal states but also about the path required to reach them. A target that is safe in isolation may be unsafe if the transition passes through a constrained region.

The distinction between commanded, actual, and estimated state is particularly important. The executor records what was requested, the dynamics layer determines what physically occurred, and the next perception estimates the resulting state. Disagreement among these values may indicate normal inertia, model error, execution failure, or an external disturbance. Keeping them separate provides the evidence needed to diagnose control performance and prevents a requested target from being reported prematurely as an accomplished result.

Handoff illustrates changing authority and scope. When an aircraft leaves the terminal environment, it is not merely deleted; responsibility conceptually transfers to another controller. A production architecture would record the time, state, destination authority, and acknowledgment of that transfer. This prevents two controllers from acting simultaneously or neither controller from recognizing responsibility. The same pattern applies whenever an object crosses organizational, geographic, or system boundaries.

Financial dynamics similarly require monitoring throughout the transition from current to desired exposure. An order can move price, consume liquidity, alter risk, or remain partially unfilled while conditions change. Handoff may occur between an allocation engine and an execution algorithm, across trading sessions, or from automated handling to a human trader. Layer 10 shows why autonomous control must model latency, inertia, and responsibility transfer rather than assuming that a valid decision produces an immediate and complete result.

Layer 11: contingencies and exogenous shocks

Cell 21 introduces scheduled environmental shocks. WX1 activates at tick 60. WX2 activates at tick 105. WX1 intensifies at tick 175 by increasing severity and radius. WX1 dissipates at tick 225. Wind evolves separately and may change simplified runway modes.

These events are not errors and they are not chosen by the controller. They are changes in the environment. Their architectural purpose is to force the system to reconsider a plan that may have been reasonable under a previous state.

Layer 11: controlled environmental shocks, Cell 21.

A robust autonomous system should therefore be judged partly by how it responds to state discontinuities. The question is not whether the initial plan was optimal. It is whether the architecture detects that the premises of the plan have changed and can reconstruct a new decision context.

Contingencies test whether the architecture is genuinely closed loop. An open-loop system would continue to execute the earlier plan despite altered weather or infrastructure. The notebook instead permits environmental events to modify the world before perception and judgment are rebuilt. The resulting action may be different not because the previous reasoning was erroneous, but because the decision problem itself has changed. This distinction is central to evaluating autonomous behavior fairly.

Scenario design should include both anticipated and novel combinations of stress. Scheduled weather activation tests a known type of disturbance at a controlled time, enabling repeatable comparisons across policies. Additional experiments could vary severity, duration, location, simultaneous runway changes, or failures in perception and communication. Repeated scenarios reveal whether the controller responds proportionately, recovers after conditions improve, and avoids unstable cycles of overreaction.

Contingency policy must also define priorities when normal objectives become incompatible. Under stress, the system may sacrifice efficiency, throughput, or schedule adherence to preserve harder constraints. These priorities should exist in deterministic governance and operational procedures rather than being improvised by a language model. Supervisory judgment can help interpret the system-wide implications, but it should not silently redefine the institution's risk appetite during an event.

The financial analogue extends beyond market volatility. A venue outage, delayed position feed, breached collateral threshold, restricted instrument update, cyber incident, or loss of model service can each invalidate part of the current plan. The correct response may include reducing authority, canceling outstanding actions, switching data sources, preserving liquidity, or escalating to human control. Layer 11 demonstrates that resilience is the capacity to reconstruct safe operation under changed premises, with the event and response fully recorded for later analysis.

TRANSFER PRINCIPLE 6 – TREAT MARKET SHOCKS AS STATE CHANGES, NOT AS MODEL FAILURES

Trading contingencies include volatility jumps, exchange halts, data-feed failures, borrow recalls, liquidity evaporation, correlation breaks, macro announcements, margin changes, and operational outages. The correct architecture does not defend an obsolete plan; it rebuilds state and re-enters the governed loop.

Layer 12: the closed autonomous feedback loop

Cell 22 is the architectural center of the notebook because it composes the layers into a cycle. Each call to `sim_tick()` applies contingencies, advances weather, updates wind and infrastructure, builds the current packet, determines whether supervisory judgment should be invoked, obtains proposals, executes validated actions, advances aircraft dynamics, and records the resulting state.

The order is deliberate. Environmental changes occur before the decision packet is constructed. Execution occurs before dynamics. Telemetry is calculated after the world responds. The next invocation therefore begins from the state that actually resulted from the previous cycle.

The supervisory trigger combines periodic calls, event-triggered calls, and conflict-triggered calls. This creates a two-speed architecture: fast deterministic control runs continuously, while generative judgment is invoked when time or state makes it useful.

Layer 12: feedback and the closed autonomous control cycle, Cell 22.

Feedback-control theory formalizes why this matters. A controller cannot remain intelligent if it does not measure the consequence of its own action [2]. In this notebook, feedback is not merely a numerical correction. It is the mechanism by which perception, forecast, context, and judgment are rebuilt.

The loop also establishes causally ordered evidence. At each tick the system can distinguish the state before intervention, proposals considered, actions authorized, target changes applied, world evolution, and measured state afterward. This sequence supports attribution even when several aircraft and disturbances interact. Without it, an operator might observe improvement or deterioration without knowing whether it resulted from a controller action, ordinary dynamics, or an exogenous event.

Trigger design determines when supervisory intelligence enters the cycle. Purely periodic invocation is predictable but may react slowly to sudden risk. Purely event-driven invocation can respond quickly but may become noisy or expensive when events cluster. Combining time, event, and conflict triggers balances these considerations. Hysteresis, cooldown intervals, and materiality thresholds can further prevent repeated judgment calls caused by small fluctuations around a boundary.

Closed-loop behavior must be assessed for stability, not only immediate effectiveness. A speed adjustment that removes one warning may create another, and repeated opposing commands may generate oscillation. Telemetry should therefore measure intervention frequency, reversal rates, persistent conflicts, queue evolution, and recovery time after shocks. The objective is a system that converges toward acceptable conditions while using proportionate control effort.

In finance, feedback includes acknowledgments, fills, transaction costs, updated positions, profit and loss, risk, liquidity, and market response. These realized outcomes must update the next decision packet rather than being treated solely as end-of-day reporting. A slower supervisory layer may then revise capital allocation or execution posture while fast controls continue to enforce limits. Layer 12 is the point at which all forms of intelligence become an adaptive process: the system learns operationally by observing the consequences of governed action and re-entering the cycle from the world that actually exists.

Layer 13: observability as a control requirement

Cell 23 creates a radar-style system view. Airports, active weather, active aircraft, callsigns near airports or weather, altitude, and runway mode are made visible. Cell 22 simultaneously maintains telemetry and an audit log.

Observability is not decoration. An autonomous system that cannot explain its current state, the source of a proposal, the acceptance or rejection of that proposal, and the path from decision to outcome is difficult to supervise and difficult to improve. The notebook therefore surrounds the control loop with human-facing evidence.

Layer 13: radar visualization and system observability, Cell 23.

There are at least three distinct forms of observability in the experiment:

- **State observability:** what is happening in the world?
- **Decision observability:** which reasoning source proposed what?
- **Governance observability:** which proposal was accepted or rejected, and why?

These distinctions are directly transferable to financial systems, where P&L visibility alone is inadequate. A trading organization must also know risk state, order state, control state, decision lineage, and exceptions.

Observability requires both real-time views and durable records. The radar view supports immediate comprehension of position, weather, traffic, and infrastructure, while telemetry permits trends to be inspected across time. The audit log preserves discrete decisions and governance outcomes. These artifacts serve different audiences: an operator needs rapid situational awareness, an engineer needs diagnostic detail, a risk manager needs exposure and control status, and an auditor needs immutable lineage from evidence to action.

Useful observability includes provenance. A displayed conflict count should identify the prediction version and state timestamp from which it was computed; an intervention should identify whether it came from Claude or fallback; and a rejection should name the exact rule that blocked it. Provenance prevents dashboards from becoming persuasive but unverifiable summaries. It also enables incident reconstruction when the current display has moved beyond the state in which a decision occurred.

The system must expose uncertainty and degraded conditions, not merely nominal outputs. Stale observations, missing fields, delayed model responses, fallback activation, validator exceptions, and incomplete handoffs should be visible as first-class states. Otherwise an apparently normal visualization can conceal a loss of capability. Alerting should be prioritized and rate-limited so that critical governance failures are not buried beneath routine events.

Observability also creates accountability without granting direct control. Human supervisors can inspect the state, understand why intervention occurred, and determine whether escalation or policy revision is needed. Their feedback can improve schemas, thresholds, candidate generation, and prompts while preserving the formal execution boundary. This supports governance as an ongoing institutional process rather than a one-time certification exercise.

For financial systems, the equivalent evidence spans market-data health, forecasts, positions, orders, fills, limits, exceptions, model versions, approvals, and realized costs. A profit number cannot show whether an unauthorized risk was taken, whether a favorable outcome was accidental, or whether a valid decision was poorly executed. Layer 13 makes control quality visible across state, decision, governance, and outcome, enabling the organization to supervise autonomy and learn from it.

Layer 14: the integrated experiment

Cell 24 resets the full simulator and runs it from tick 1 through `MAX_TICKS`. Airport state, weather, random seeds, fleet, telemetry, and audit history are reset before execution. The integrated experiment can therefore be rerun without inheriting hidden state from a previous trial.

The run uses Claude when `USE_CLAUDE=True` and a client is available. Otherwise it falls back deterministically. Periodic snapshots and event-triggered output expose important transitions. The experiment ends with a telemetry DataFrame, time-series plots, a final airspace view, and a governance audit summary.

Layer 14: integrated closed-loop experiment, Cell 24.

The integrated run is not primarily a performance contest. Its function is architectural verification. Every layer must cooperate: state must be reset, the world must evolve, perception must remain consistent, prediction must run, congestion must be computed, context must be constructed, judgment must obey the interface, validation must gate execution, dynamics must produce the next world, shocks must enter, and audit history must remain coherent.

Integration reveals failures that isolated component tests cannot. A predictor may be correct when tested alone but receive stale observations in the full loop. A validator may enforce each rule correctly yet reject every proposal because candidate generation and context serialization use different value formats. Telemetry may record actions but associate them with the wrong tick. Running the complete experiment tests interfaces, ordering, state ownership, and recovery behavior as well as the local logic of each layer.

Reset behavior is a particularly important part of experimental validity. Airports, weather, fleet objects, counters, random state, audit records, and telemetry must all return to a known baseline. Partial reset can create hidden dependencies in which later runs benefit from prior state or reproduce results only in a particular notebook execution order. The integrated cell therefore acts as an executable experimental protocol, not merely a convenience for starting the simulation.

Evaluation should compare more than whether the run completes. Relevant measures include predicted and realized conflict conditions, congestion duration, number and type of proposals, acceptance and rejection rates, fallback use, intervention reversals, handoff completion, response to scheduled shocks, latency, and audit completeness. Comparisons across seeds and scenario variants can reveal whether apparent success is robust or dependent on a favorable initial configuration. The synthetic setting makes these tests repeatable while keeping the emphasis on architecture.

The experiment also supports controlled ablation. Researchers can disable supervisory judgment, alter trigger policies, change candidate menus, tighten validation envelopes, remove a forecast feature, or substitute another model while holding other layers constant. The resulting differences help identify which capability creates value and which governance mechanism contains risk. This is more informative than comparing opaque end-to-end agents whose internal responsibilities differ.

Transferred to finance, an integrated experiment should run through historical replay, simulated execution, shocks, feed degradation, venue outages, model failure, risk-limit changes, and recovery. It should verify that every capital movement is traceable to current evidence and valid authority. Layer 14 closes the pedagogical argument: multidimensional intelligence becomes credible only when perception, prediction, system analysis, bounded judgment, governance, execution, dynamics, resilience, feedback, and observability operate coherently as one reproducible system.

A formal view of the architecture

The notebook can be written as a governed state-space system. Let x_t denote the complete world state and o_t the deterministic observation:

$$o_t = H(x_t).$$

The prediction layer produces a forward representation,

$$\hat{x}_{t+h} = P_h(o_t),$$

and the system-analysis layer produces aggregate interaction measures,

$$s_t = A(o_t, \hat{x}_{t+h}).$$

The candidate generator and context constructor produce

$$\mathcal{C}_t = C_A(x_t), \quad c_t = C_S(o_t, \hat{x}_{t+h}, s_t, \mathcal{C}_t).$$

Supervisory judgment produces a proposal set,

$$p_t = J(c_t).$$

The validator maps proposals into executable authority,

$$a_t = V(p_t, x_t, \mathcal{C}_t).$$

The world then evolves under executed action and exogenous disturbance,

$$x_{t+1} = F(x_t, a_t, \varepsilon_{t+1}).$$

This decomposition is valuable because it exposes where different failure modes reside. A perception error is not the same as a forecast error. A forecast error is not the same as an inadmissible action. A good proposal rejected by an overly restrictive validator is a governance-design problem. A valid target that produces an unexpected outcome may be a dynamics-model problem. By separating layers, the architecture makes diagnosis possible.

The model can also be represented as two nested loops. The **fast loop** updates state, forecasts, rules, execution, and dynamics. The **slow loop** interprets the structured state and proposes higher-level interventions. This is analogous to hierarchical and supervisory control architectures in which different controllers operate at different time scales.

From airspace to finance: the multi-market multi-asset control problem

The ultimate value of the experiment is not aviation. It is transfer. A complex trading organization already resembles the airspace system in several structural ways. Many instruments move simultaneously. Actions interact through portfolio constraints and market impact. Multiple venues share liquidity. External shocks change the state without permission. Some decisions must occur in milliseconds; others require slower interpretation. Execution is subject to hard rules. Risk limits, legal constraints, exchange rules, and internal governance matter independently of the quality of the alpha model.

The architectural mapping is therefore natural.

Layer	Airspace sandbox	Financial control system
1	Compute/API/reproducibility	Market-data stack, model registry, credentials, configuration, clock synchronization
2	Airports, aircraft, weather, wind	Markets, venues, instruments, positions, cash, collateral, borrow, market status
3	Aircraft observation table	Normalized market state, positions, exposures, liquidity, order state
4	Short-horizon trajectory prediction	Volatility, spread, impact, fill, correlation, liquidity and scenario forecasts
5	Congestion and sequencing	Crowding, factor concentration, portfolio pressure, venue capacity, order priority
6	Candidate actions and validator	Risk limits, leverage caps, participation bands, restricted lists, price collars, kill switches
7	Situation packet	Compact supervisory decision packet containing risks, forecasts, constraints and candidates
8	Claude supervisory judgment	Regime interpretation, cross-asset prioritization, strategy reweighting, exception reasoning
9	Execution targets	Approved orders, hedges, cancellations, participation-rate changes, strategy enable/disable
10	Aircraft dynamics	Fills, slippage, latency, market impact, position and P&L evolution
11	Weather/wind shocks	News, halts, outages, volatility spikes, funding changes, liquidity breaks
12	Feedback loop	Continuous mark-to-market, risk recomputation, order feedback, re-forecast and re-decision
13	Radar/telemetry/audit	Risk dashboard, order blotter, model lineage, control-state and governance logs
14	Integrated simulation	Full multi-market multi-asset autonomous trading experiment

The financial world state

A trading world state can be represented as

$$x_t^F = (M_t, Q_t, C_t, L_t, R_t, O_t, V_t, G_t),$$

where M_t is market state, Q_t positions, C_t cash and collateral, L_t liquidity, R_t risk measures, O_t outstanding orders, V_t venue/connectivity state, and G_t governance constraints. This state is richer than a price vector because control decisions depend on institutional position and market capacity, not only on expected return.

Perception and prediction should remain heterogeneous

The airspace notebook does not ask Claude to compute distance, trajectory geometry, or pairwise conflict thresholds. The financial analogue is equally important. Market microstructure statistics, portfolio Greeks, factor exposures, VaR/ES, liquidity measures, and forecast models should remain explicit analytical components. A language model may interpret their joint significance, but it should not replace calculations that are already structured and testable.

Optimal-execution and market-making research makes the same distinction from another direction: execution quality depends on dynamics, market impact, inventory, and timing, not merely on a directional view [10; 11; 12]. A

judgment layer should therefore reason *over* execution models rather than substitute for them.

The most important financial transfer is the harness

Suppose a supervisory model concludes that a portfolio should reduce risk in a technology selloff. That conclusion is not yet an executable order. The harness might generate a finite menu:

- reduce exposure by 2, 5, or 8 percent;
- hedge beta using one of a small set of approved index futures;
- reduce participation rate in selected venues;
- cancel passive orders outside a new price collar;
- do nothing and request human review.

Each candidate can be constructed from current risk, liquidity, compliance, and capacity constraints. A deterministic validator can then reject proposals that breach leverage, notional, participation, inventory, restricted-list, short-sale, market-hours, or concentration rules.

The critical institutional principle is the same as in Cell 5:

Judgment proposes; governance authorizes; execution acts.

Multi-timescale intelligence

The two-loop structure becomes even more valuable in markets. A possible hierarchy is:

- **milliseconds to seconds:** data validation, order-state updates, hard risk controls, execution algorithms;
- **seconds to minutes:** portfolio risk, liquidity, cross-asset interaction, anomaly detection;
- **minutes to hours:** supervisory judgment, regime interpretation, strategy allocation, exception handling;
- **hours to days:** human committees, model approval, parameter governance, post-trade review.

No single model should be forced to operate at all four frequencies. Fast layers require deterministic latency and high reliability. Slower layers can use richer context and broader semantic reasoning. This allows the institution to add generative judgment without putting an LLM into the market-data or order-routing critical path.

From autonomous trading to governed autonomous finance

The financial end-state is not an “AI trader” that replaces an algorithmic stack. It is a **governed autonomous financial control system** in which existing automation gains a supervisory intelligence layer.

A useful target architecture is:

Markets → Perception → Forecasts → Portfolio/System Analysis
→ Decision Packet → Judgment → Risk/Compliance Harness
→ Execution → Fills and P&L → Feedback.

The model can be upgraded as model technology improves, but the permission system remains institutional. This is precisely the direction implied by modern AI risk-management thinking: trustworthy deployment depends on the complete socio-technical system, not only on the predictive quality of the model [6].

THE FINANCIAL RESEARCH QUESTION

How much additional value can bounded generative judgment create when it is inserted *above* an already autonomous multi-market, multi-asset trading engine, while all execution authority remains inside deterministic risk, compliance, and market-microstructure controls?

Research agenda

The notebook is intentionally transparent and small enough to teach. A research-grade extension should deepen each layer without destroying the separation among them.

First, the world model can become more realistic while remaining synthetic: richer interaction networks, capacity constraints, stochastic delays, correlated shocks, and heterogeneous information quality. Second, prediction can evolve from linear projection to probabilistic distributions. Third, the system-analysis layer can use graph measures, queueing models, and endogenous congestion. Fourth, the context layer can formalize attention allocation and uncertainty.

Fifth, the harness can become stateful. Instead of static numerical envelopes, permissions can depend on accumulated risk, prior interventions, confidence, unresolved exceptions, and time since last human review. Sixth, the judgment layer can compare multiple candidate reasoners or use a committee of models while preserving one deterministic execution gate. Seventh, the observability layer can support counterfactual replay: what would have happened if a rejected proposal had been accepted?

For finance, the most interesting next experiment is a synthetic multi-market, multi-asset trading laboratory in which deterministic alpha, execution, and risk engines already operate autonomously. The generative layer should be added only after that baseline exists. The experiment should then measure the marginal value of judgment: Does the supervisory layer improve regime adaptation, portfolio coordination, exception handling, or recovery from shocks? At what cost in latency, turnover, or complexity? Which interventions should remain unavailable even when model reasoning appears strong?

Limitations

Several limitations are deliberate. The simulator does not model real airspace procedures or safety standards. Its synthetic units do not map directly to nautical miles, feet, or knots in an operational sense. The conflict detector is a pedagogical threshold mechanism, not a certified separation tool. The wind/runway logic is a simplified environmental-state switch. The priority policy is invented for teaching.

The model interface also remains simple. Claude receives a text/JSON situation packet, not a formally verified symbolic policy. The deterministic fallback is intentionally modest. The notebook does not optimize a global objective over all aircraft. It demonstrates architecture rather than operational optimality.

These limitations strengthen rather than weaken the transfer lesson. The monograph does not claim that the specific aviation rules should be exported to finance. It claims that the **separation of functions** should be exported: state construction, prediction, system analysis, bounded judgment, executable governance, execution, dynamics, contingencies, feedback, and observability.

Conclusion: the controlled object is the system

The airspace experiment shows why autonomy becomes qualitatively different when many agents interact. The problem is no longer to generate a good action for one object. It is to maintain a coherent state of the whole system, anticipate interactions, allocate attention, interpret competing considerations, authorize only admissible actions, observe real consequences, and repeat the process as the environment changes. A recommendation may be locally sensible and globally destabilizing; a valid action may cease to be appropriate after a shock; and an intervention aimed at one entity may change the feasible choices of many others. The unit of intelligence must therefore expand from an isolated decision to the organization of the entire control loop.

The central conclusion is that operational intelligence is **multidimensional**. Perception, prediction, judgment, governance, execution, feedback, traceability, and reproducibility are not interchangeable labels for the same capability. They are distinct requirements that must work together. Perception without judgment can describe a problem without resolving it. Judgment without a harness can generate actions that the system cannot safely interpret. A harness without governance may format a proposal but fail to establish whether it is permissible. Governance without reliable execution can authorize behavior that never occurs as intended. Execution without feedback cannot determine whether the intervention succeeded. Feedback without traceability cannot explain the path to the outcome, and traceability without reproducibility cannot support disciplined comparison or scientific learning.

The simulator's fourteen layers make these relationships concrete. The world model establishes the entities, infrastructure, and disturbances that constitute the environment. Perception creates a structured present. Prediction turns that present into possible futures. System analysis reveals congestion and interaction at the population level. Context construction determines what evidence supervisory judgment receives. Claude contributes bounded interpretation, but the action generator and deterministic validator restrict the space of possible intervention. Execution changes targets; dynamics govern realized movement; contingencies modify the environment; and feedback reconstructs the next state. Telemetry and audit records make the process inspectable. Reproducible initialization and centralized configuration make experiments comparable. Intelligence resides in the coordination of all these functions, not in the most rhetorically impressive component.

The strongest architectural lesson is consequently the separation of reasoning from authority. Claude can contribute judgment without controlling the aircraft. The candidate generator defines what can be considered. The validator decides what can be authorized. Execution modifies targets. Dynamics determine what actually happens. Feedback creates the next decision problem. If Claude becomes unavailable, the architecture continues with deterministic fallback. If Claude becomes more capable, its greater capability does not automatically enlarge its permission set. This decoupling permits model development, operational authorization, and institutional accountability to proceed on different schedules. A model can be replaced or improved while governance remains stable; a policy boundary can be revised without rewriting the model; and an incident can be analyzed by examining which layer failed to perform its defined role.

Harnessing is fundamental to this separation. In this study the harness is not simply a wrapper around a model call. It is the mechanism that converts broad reasoning into a finite, typed, and reviewable proposal. It supplies a constrained action vocabulary, constructs candidate interventions, provides relevant context, parses the response, rejects malformed outputs, applies deterministic limits, and invokes fallback behavior where necessary. In institutional terms, it functions like a constitution for machine judgment: it defines what questions the model may answer, what actions it may propose, what evidence it must use, and which independent authority decides whether a proposal can proceed. Governance becomes executable because these boundaries are implemented in the control path rather than stated only in policy documents.

Traceability gives this governance retrospective force. For any consequential action, an operator should be able to reconstruct the perceived state, forecast horizon, system-level indicators, candidate set, model response, validation outcome, executed target, and realized consequence. This reconstruction supports more than compliance. It allows the organization to locate failure precisely. A missed risk may originate in incomplete perception, a weak predictor, an omitted interaction, an unsuitable candidate set, poor judgment, an incorrect rule, an execution defect, or an external shock. Treating all adverse outcomes as "model errors" would conceal these distinctions and prevent useful improvement.

Reproducibility gives the architecture prospective discipline. Fixed seeds, resettable environments, versioned configurations, centralized model access, and explicit assumptions allow a researcher to rerun scenarios and compare alternatives. In production finance, exact market histories cannot be recreated in real time, but decision conditions can still be captured and replayed through event data, configuration snapshots, model versions, and simulated execution. Reproducibility therefore means preserving enough of the decision environment to test counterfactuals: whether a different judgment policy would have acted, whether a stricter limit would have blocked the action, or whether the same architecture behaves consistently under related shocks.

The transfer to finance follows from these structural insights. Algorithmic trading already contains perception, prediction, execution, dynamics, and feedback. Market-data systems observe prices, volumes, order books, and reference information. Models forecast returns, volatility, liquidity, and risk. Portfolio and execution engines transform desired exposures into orders. Venues and other participants determine fills and market impact. Position, profit-and-loss, and risk systems observe the result. The opportunity is not to discard this mature machinery in favor of an LLM. It is to add a governed judgment layer that can interpret the state of the whole investment system—across assets, venues, strategies, mandates, liquidity conditions, and shocks—while leaving hard authority in deterministic mechanisms designed for speed, auditability, compliance, and control.

What transfers is not a mapping of aviation rules to trading rules. Systems-level learning transfers the decomposition of the problem, the interfaces among functions, and the discipline of closed-loop control. In airspace, the system may prioritize trajectories, capacity, separation, and handoff. In finance, it may coordinate exposures, cash, leverage, liquidity, market impact, and mandate constraints. The variables and consequences differ, but both environments require the controller to reason about interacting entities under shared constraints and uncertain evolution. Both need to distinguish a proposed objective from an admissible action, and an admissible action from a realized outcome.

This perspective changes how investment intelligence should be evaluated. A high-quality forecast is only one dimension. A robust investment-control system must also know which accounts can act, how much capital is available, which exposures are correlated, how crowded a trade has become, what liquidity is realistically accessible, which rules are binding, how quickly an intervention can be reversed, and when uncertainty requires escalation rather than action. It must record the evidential and governance path from observed market state to capital movement. It must then measure whether the realized fills, costs, exposures, and downstream risks correspond to the intended decision. The controlled object is not the individual order or even the individual portfolio; it is the interconnected flow of information, authority, capital, and risk.

The physical character of air-traffic control makes this systems logic especially visible. Aircraft have inertia, occupy shared space, face environmental disturbances, and cannot instantaneously realize a command. Financial positions also exhibit a form of operational inertia: liquidity limits adjustment speed, market impact changes execution cost, crowded trades couple participants, and portfolio constraints make one allocation dependent on others. This analogy should not be overstated, but it supports a valuable lesson. Decisions act through a world with resistance, latency, interaction, and uncertainty. A controller that ignores those dynamics may produce elegant targets that cannot be reached safely or economically.

Accordingly, the appropriate research question is not whether a language model can produce an attractive trade idea or a plausible air-traffic instruction. It is whether the complete governed system perceives accurately, reasons usefully, refuses impermissible behavior, executes reliably, adapts to consequences, degrades gracefully, and leaves evidence sufficient for explanation and learning. Performance must be assessed across these dimensions, including false interventions, missed interventions, latency, rule violations, execution quality, resilience to shocks, stability under repeated runs, and operator comprehension. This wider evaluation recognizes that autonomous intelligence is an institutional and engineering property as much as a modeling property.

The transfer from airspace to markets can therefore be stated in one sentence:

Add judgment to autonomy without surrendering governance.

That design principle is broader than aviation and broader than trading. It is a general architecture for institutions that want autonomous systems to become more intelligent without becoming less controllable. The most defensible path toward greater autonomy is not to concentrate perception, judgment, authority, and execution in a single opaque component. It is to compose specialized capabilities within explicit boundaries, preserve human and institutional authority over those boundaries, and treat every action as part of an observable feedback process.

The ultimate contribution of the experiment is therefore a framework for learning across domains. A physical operational system can teach finance how to represent interacting state, separate supervisory reasoning from hard authority, engineer bounded action spaces, incorporate dynamics and shocks, and build traceable feedback. Finance can instantiate those principles with its own data, risk measures, mandates, execution mechanisms, and legal obligations. The transferred knowledge lies at the level of system organization. When that distinction is respected, learning from airspace can improve the control of investment flows and decisions without importing false equivalences or surrendering domain expertise.

Autonomy, in this account, is neither a single model nor a sequence of automated tasks. It is a governed capacity to observe, judge, act, verify, explain, and learn over time. Its intelligence is multidimensional; its authority is deliberately bounded; and its success depends on the integrity of the complete loop. That is the architecture required to add judgment to complex systems while preserving control.

Appendix: Notebook-to-Layer Map

Layer	Cell(s)	Notebook content
1	2–4	Dependencies, reproducibility, global configuration, Claude interface
1 / Architecture	5	Closed-loop governed autonomous-control narrative; fast and slow loops; authority boundaries
2	6–10	Airports, aircraft performance, 60-aircraft fleet, moving weather, wind and runway modes
3	11	Perception DataFrame
4	12	Short-horizon prediction and synthetic conflict detection
5	13	Congestion and sequencing
6	14–16	Harness concept, bounded action generation, deterministic validator
7	17	Compact world-state packet
8	18	Claude bounded judgment with deterministic fallback
9	19	Execution after validation
10	20	Aircraft dynamics and handoff
11	21	Controlled environmental shocks
12	22	Complete closed-loop simulation tick
13	23	Radar-style observability
14	24	Integrated experiment
–	25	Conclusion: from an autonomous agent to a governed autonomous system

Appendix: Source Notebook and Figure Corpus

The primary source for the architecture, code behavior, synthetic numerical examples, cell mapping, and pedagogical narrative in this monograph is:

```
@TOPIC_784 SELF DRIVING ATC MMMX.ipynb
```

All layer infographics included in the monograph were supplied with the notebook corpus and correspond to the sections in which they appear. Layer 1 includes two figures: the setup/initialization infographic for Cells 2–4 and the architecture infographic specifically associated with the Cell 5 narrative.

References

- [1] Reynoso, A. (2026). *TOPIC_784 SELF DRIVING ATC MMMX.ipynb*. Jupyter/Google Colab notebook. Primary source for the simulator architecture, code, synthetic examples, and layer narratives in this monograph. Referenced in the project as `@TOPIC_784 SELF DRIVING ATC MMMX.ipynb`.
- [2] Åström, K. J., and Murray, R. M. (2008). *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press.
- [3] Endsley, M. R. (1995). “Toward a Theory of Situation Awareness in Dynamic Systems.” *Human Factors*, 37(1), 32–64. doi:10.1518/001872095779049543.
- [4] Parasuraman, R., Sheridan, T. B., and Wickens, C. D. (2000). “A Model for Types and Levels of Human Interaction with Automation.” *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 30(3), 286–297. doi:10.1109/3468.844354.
- [5] Leveson, N. G. (2011). *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.
- [6] National Institute of Standards and Technology (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1.
- [7] Russell, S. J., and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson.
- [8] Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.
- [9] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). “ReAct: Synergizing Reasoning and Acting in Language Models.” *International Conference on Learning Representations (ICLR)*.
- [10] Almgren, R., and Chriss, N. (2001). “Optimal Execution of Portfolio Transactions.” *Journal of Risk*, 3(2), 5–39.
- [11] Avellaneda, M., and Stoikov, S. (2008). “High-Frequency Trading in a Limit Order Book.” *Quantitative Finance*, 8(3), 217–224.
- [12] Cartea, Á., Jaimungal, S., and Penalva, J. (2015). *Algorithmic and High-Frequency Trading*. Cambridge University Press.
- [13] Hasbrouck, J. (2007). *Empirical Market Microstructure: The Institutions, Economics, and Econometrics of Securities Trading*. Oxford University Press.
- [14] Bertsekas, D. P. (2017). *Dynamic Programming and Optimal Control*, 4th ed. Athena Scientific.

Source note. The aviation mechanisms in this monograph are described only as synthetic educational constructs. External references are used to situate the architecture within control, human-automation, AI-governance, and market-microstructure literature; they are not sources for the notebook’s synthetic aviation rules.